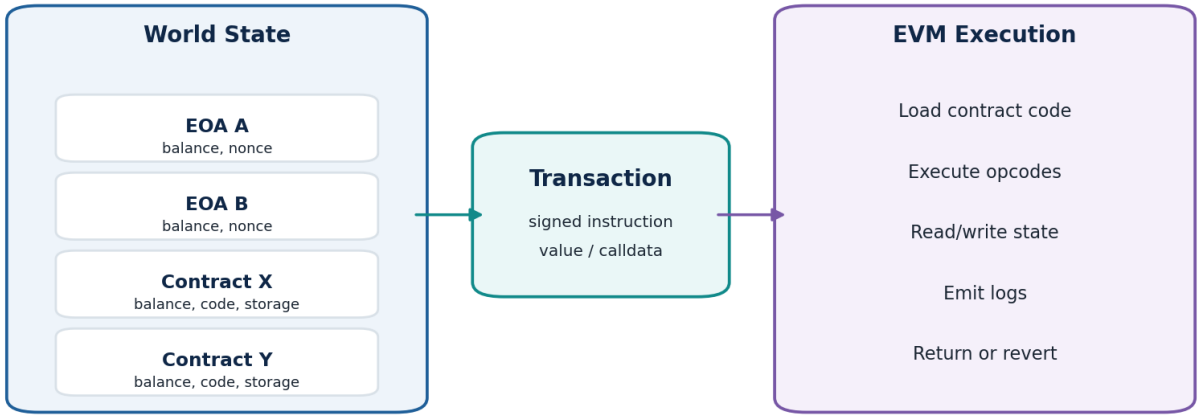
20-DAY BLOCKCHAIN OUTSOURCING MASTER COURSE

Day 3 - Ethereum & the EVM

Complete 60-Minute Beginner Lecture | Accounts, Transactions, Gas, Calldata, Bytecode, ABI, Events & Debugging

Ethereum Mental Model: Transactions Transform Global State



Result: a new valid world state + transaction receipt/logs

Day 3 outcome

By the end of this class, students should be able to explain Ethereum as a state machine, distinguish EOAs from contract accounts, read a transaction and receipt, understand nonce and gas, describe how the EVM uses stack/memory/storage/calldata, explain bytecode/ABI/function selectors/events, and follow a basic transaction-debugging workflow.

Course role	Focus today	Professional payoff
Beginner foundation	Understand what Ethereum actually executes	Stop treating wallets/explorers as magic
Developer foundation	Connect Solidity concepts to EVM behavior	Write and debug smarter contracts later
Freelancer foundation	Read public transaction evidence	Diagnose client failures methodically

Safety rule

Use public transaction hashes, public addresses and test networks for exercises. Never ask a client for a seed phrase or private key to inspect a transaction.

60-Minute Teaching Plan

Time	Topic	What students should leave with
0-5 min	Recap + Ethereum mental model	Transactions transform a shared global state
5-13 min	Accounts & world state	EOA vs contract; balance, nonce, code, storage
13-23 min	Transaction anatomy	chain ID, nonce, to, value, calldata, signature
23-31 min	Gas & fee model	gas limit, gas used, base fee, priority fee, failure behavior
31-41 min	EVM execution model	opcodes; stack, memory, storage, calldata
41-49 min	Bytecode, ABI & function calls	compiler artifacts, selectors, ABI encoding
49-55 min	Events, logs & receipts	how applications and explorers observe execution
55-60 min	Debugging lab + quiz	systematic first-response workflow

Instructor preparation

- Open an Ethereum block explorer in advance and select one successful contract interaction plus one failed transaction.
- Have a simple Counter contract ready to illustrate storage and events.
- Use a whiteboard to keep one recurring diagram: EOA -> transaction -> contract -> EVM -> state/logs.
- If demonstrating JSON-RPC, use read-only methods; students do not need real funds today.

Day 2 recap questions

- What is the difference between a private key and an address?
- What does a digital signature prove?
- Why is a seed phrase more sensitive than a local wallet password?
- What is the difference between signing a message and sending a transaction?

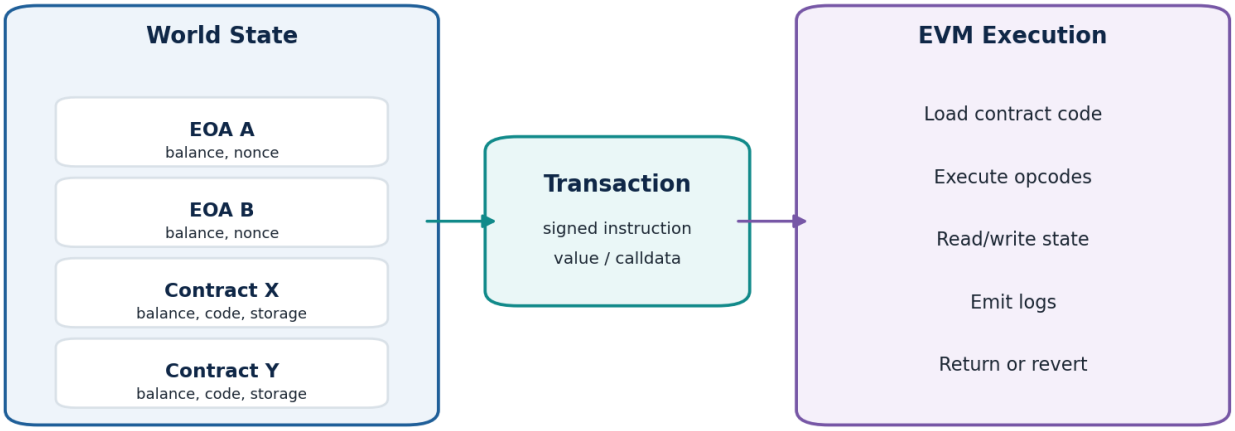
Bridge from Day 2 to Day 3

Yesterday we learned who can authorize an action. Today we learn what Ethereum does after the action is signed.

0-5 min - Ethereum as a State Machine

A useful developer mental model is: Ethereum maintains a shared world state. A valid transaction is an authorized instruction. Nodes execute that instruction according to protocol rules. If execution succeeds, the network arrives at a new state; if execution reverts, state changes from that execution are rolled back, although the transaction can still consume gas.

Ethereum Mental Model: Transactions Transform Global State



Result: a new valid world state + transaction receipt/logs

1. What “state” means

- Account balances
- Account nonces
- Contract bytecode associated with addresses
- Persistent contract storage such as token balances, ownership, staking totals or configuration

Simple sentence to memorize

Ethereum = a replicated state machine. Transactions are inputs; EVM execution computes valid state transitions.

2. Read vs write

Action	Example	Transaction?	Gas paid by caller?
Read	balanceOf(user)	No state-changing transaction when called locally via eth_call	No on-chain fee for the read itself
Write	transfer(to, amount)	Yes	Yes, if submitted on-chain
Write	stake(amount)	Yes	Yes
Read	owner()	No state-changing transaction when read off-chain	No on-chain fee for the read itself

Freelancer lesson

A client saying “the contract call failed” is incomplete. First ask: was this an off-chain read (eth_call), a submitted transaction, or a transaction that was mined and reverted?

5-13 min - Ethereum Accounts & World State

3. Externally owned accounts (EOAs)

An EOA is traditionally controlled by a private key. It can initiate signed transactions. The address has a balance and a nonce. In the classic account model, an EOA does not store contract bytecode.

4. Contract accounts

A contract account is associated with EVM bytecode and persistent storage. Contract code runs when the contract is called as part of transaction execution or another contract call. A contract does not independently wake up and initiate a normal EOA-style transaction; its execution is triggered by a transaction/call path.

Property	EOA	Contract account
Control	Private-key authorization	EVM code + contract logic
Can initiate a normal signed transaction?	Yes	No, execution is triggered through calls
Code	Normally none	Runtime bytecode
Persistent storage	No contract storage	Yes
Balance	Can hold ETH	Can hold ETH

5. Account fields: beginner view

- nonce - for an EOA, tracks sent transaction sequencing; for contracts, protocol account nonce has contract-creation significance.
- balance - amount of ETH denominated internally in wei.
- code - contract bytecode for contract accounts.
- storage - persistent key/value state used by contracts.

Nonce misconception

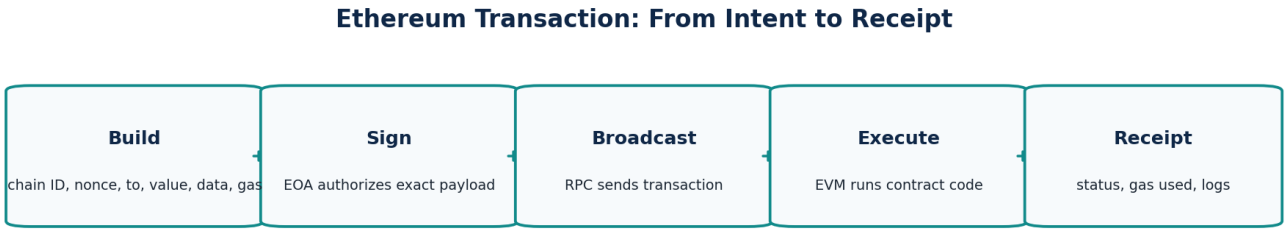
Nonce is not “the transaction number displayed by the wallet.” It is protocol state used to order transactions from an account and prevent replay/re-execution of the same sequence position.

6. Wei and ETH

```
1 ETH = 1,000,000,000,000,000,000 wei
      = 10^18 wei
```

Smart contracts and JSON-RPC values commonly use integer units. Never use floating-point math for token or ETH accounting in production code.

13-23 min - Transaction Anatomy



7. The essential transaction fields

Field	Meaning	Debugging question
chainId	Which compatible network the signed transaction targets	Is the wallet on the correct network?
nonce	Sequence position for sender	Is there a stuck/replaced nonce?
to	Recipient EOA or contract; absent for contract creation	Is this the expected contract?
value	Native ETH attached	Was ETH supposed to be sent?
data / input	Arbitrary calldata, often encoded function selector + arguments	Which function and arguments were sent?
gas limit	Maximum execution gas the transaction may consume	Was the limit sufficient?
fee fields	How much the sender is willing to pay per gas	Was pricing competitive/valid?
signature	Cryptographic authorization by sender	Does sender recovery match the expected account?

8. Calldata: where function calls begin

For a smart-contract interaction, the transaction data field typically contains ABI-encoded calldata. For a normal Solidity external function call, the first four bytes commonly identify the function, followed by encoded arguments.

```
transfer(address,uint256)
        |
Keccak-256(function signature)
        |
first 4 bytes = function selector

calldata = selector || ABI-encoded arguments
```

Important

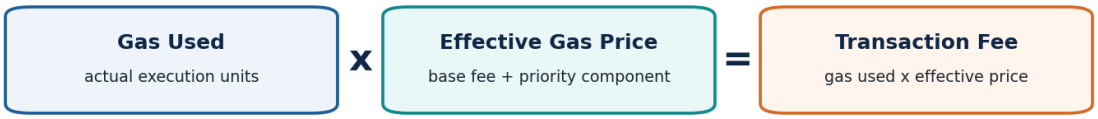
Calldata is bytes. Explorers make it human-readable only when they know the contract ABI or can infer the function signature.

9. Contract creation transaction

If the transaction has no normal recipient address (the protocol uses contract-creation semantics), the input contains creation/init code. That code executes once and returns the runtime bytecode that remains at the new contract address.

23-31 min - Gas, Fees & Failure

Ethereum Transaction Fee Mental Model (Type 2)



maxFeePerGas = your ceiling; maxPriorityFeePerGas = priority-fee ceiling.
The block base fee is protocol-defined and changes with demand.

Gas limit is a safety cap, not automatically the amount you pay for.

10. Gas is execution work, not a dollar price

EVM operations have gas costs. A transaction specifies a gas limit, which caps how much execution work it permits. Actual gas used depends on what the EVM executes: storage changes, external calls, memory expansion, logs and other operations all contribute.

11. Fee terms

Term	Beginner meaning
Gas limit	Maximum gas the transaction authorizes for execution
Gas used	Actual execution units consumed
Base fee	Protocol-defined per-gas component for a block; burned under EIP-1559 fee mechanics
Priority fee / tip	Component intended to reward block inclusion priority
maxFeePerGas	Sender ceiling for total per-gas price in a type-2 transaction
maxPriorityFeePerGas	Sender ceiling for priority component

12. Out of gas vs revert

Failure	What happened	What survives?
Revert	Contract deliberately/implicitly reverts during execution	State changes from reverted execution are rolled back; gas already consumed is not magically returned
Out of gas	Execution needs more gas than the remaining allowance	Execution fails and state changes are rolled back; consumed gas is charged
Dropped/not included	Transaction may never have been mined	No on-chain execution receipt because it was not included

Debugging habit

Do not diagnose a transaction from “Status: Failed” alone. Inspect the receipt, input, decoded function, emitted logs (if any), revert data and call trace when available.

31-41 min - Inside the EVM

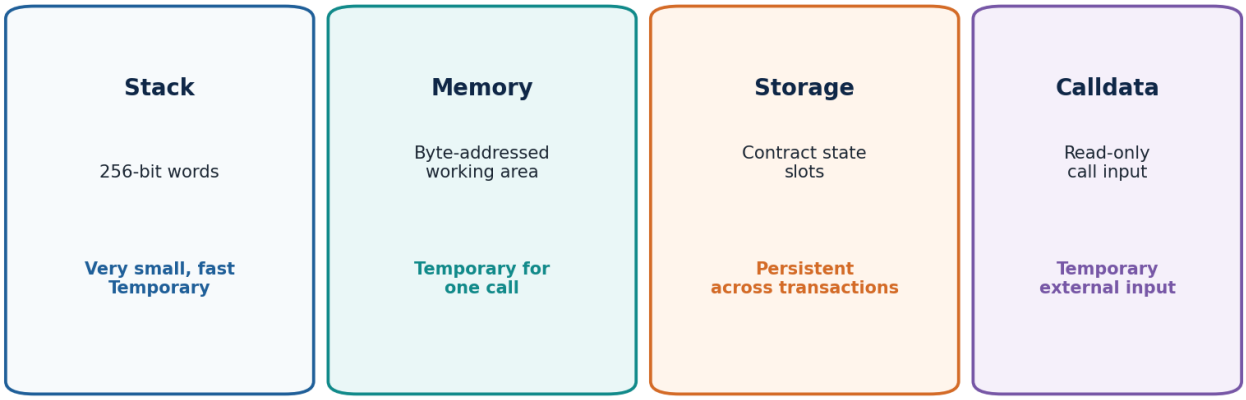
The EVM is a deterministic, stack-based virtual machine. Every Ethereum node can execute the same contract bytecode with the same transaction/block context and compute the same valid result. Solidity is not what nodes execute directly; nodes execute EVM bytecode.

13. Opcodes

Bytecode is a sequence of EVM instructions called opcodes. Examples include arithmetic, stack operations, memory operations, storage operations, control flow, calls, logging, return and revert operations.

```
Conceptual example (not exact compiler output):
PUSH1 0x01
PUSH1 0x02
ADD
...
```

EVM Data Areas: Know What Persists and What Disappears



Rule of thumb: storage changes are state changes; memory/stack/calldata are execution-time data.

14. Stack

The EVM stack holds 256-bit words used by instructions. Opcodes push values, pop values, compare them, compute with them and use them as pointers/arguments. Stack depth is bounded.

15. Memory

Memory is temporary byte-addressed working space for one message-call execution. It expands as needed and disappears after the call finishes.

16. Storage

Storage is persistent contract state. Solidity state variables ultimately map into storage slots according to compiler layout rules. Storage reads/writes are important for both gas cost and security.

17. Calldata

Calldata is the read-only input area for an external call. Solidity external function arguments are initially ABI-encoded in calldata before being decoded for use.

Memorize this

storage = persistent; memory = temporary mutable workspace; calldata = temporary read-only external input; stack = EVM operand workspace.

EVM Example - Follow One Counter Transaction

```
pragma solidity ^0.8.20;

contract Counter {
    uint256 public count;
    event Incremented(address indexed by, uint256 newCount);

    function increment() external {
        count += 1;
        emit Incremented(msg.sender, count);
    }
}
```

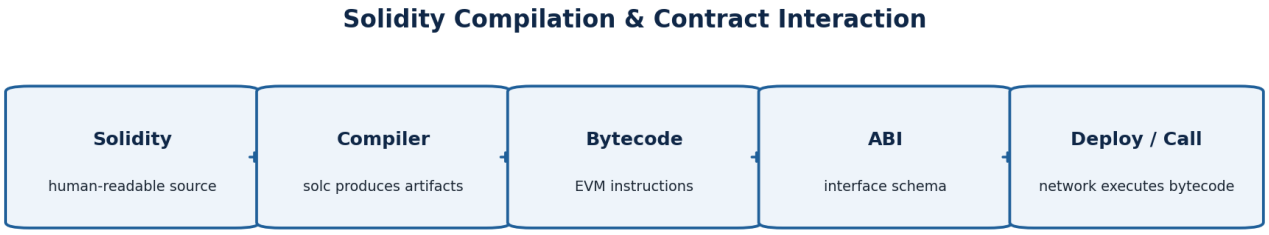
18. What happens when a user calls increment()

1. Wallet builds a transaction whose to field is the Counter address.
2. data contains the function selector for increment().
3. The EOA signs the transaction with the correct nonce and chain context.
4. A node/block builder includes the transaction in a block.
5. The EVM loads the contract runtime bytecode and dispatches execution based on calldata.
6. The contract reads count from storage, adds one, and writes the new value back to storage.
7. The LOG mechanism records the Incremented event.
8. Execution returns successfully; the receipt records status, gas usage and logs.
9. The world state now contains the updated count value.

Why this example matters

Almost every smart-contract job can be decomposed this way: transaction input -> EVM code path -> reads/writes/calls/logs -> receipt/state result.

41-49 min - Bytecode, ABI & Function Calls



19. What the Solidity compiler produces

- Creation bytecode / init code used during deployment.
- Runtime bytecode that stays associated with the deployed contract address.
- ABI JSON describing callable functions, inputs, outputs, events and errors needed by tooling.
- Metadata and other build artifacts used for verification, debugging and developer tooling.

20. ABI is a schema, not executable code

The ABI tells tools how to encode calls and decode results/logs. A frontend library such as ethers can use the ABI plus a contract address and provider/signer to construct calls.

```
[
  {
    "type": "function",
    "name": "increment",
    "inputs": [],
    "outputs": []
  },
  {
    "type": "event",
    "name": "Incremented",
    "inputs": [ ... ]
  }
]
```

21. Function selector

For standard external Solidity ABI calls, the function selector is the first four bytes of Keccak-256 of the canonical function signature. Parameter names are not part of that canonical signature; parameter types are.

```
transfer(address,uint256)   -> selector = first 4 bytes of keccak256(signature)
balanceOf(address)           -> another 4-byte selector
```

Client debugging

If the explorer says “Unknown function” or shows raw input, obtain the verified ABI/build artifact and decode the calldata. This is often faster than guessing from UI behavior.

49-55 min - Events, Logs & Transaction Receipts

22. Events are application-facing logs

Solidity events compile to EVM logging operations. Logs are stored in transaction receipts and are designed to be efficiently filtered by applications. They are not the same as contract storage: contracts generally cannot later read historical logs as if they were state variables.

```
event Transfer(address indexed from, address indexed to, uint256 value);
```

23. Topics and data

- The first topic for a non-anonymous Solidity event is commonly the hash of the event signature.
- Parameters marked indexed are placed into topics (subject to ABI/event encoding rules).
- Non-indexed parameters are ABI-encoded into the log data field.
- Applications can filter logs by contract address and topics.

24. Transaction receipt

Receipt field	Why it matters
status	Success or failure indicator for included transaction
transactionHash	Stable lookup key
blockNumber / blockHash	Where execution was included
gasUsed	Actual gas consumed by this transaction
effectiveGasPrice	Per-gas price actually applied under fee rules
contractAddress	Populated for successful contract creation
logs	Events/log records emitted during successful execution paths

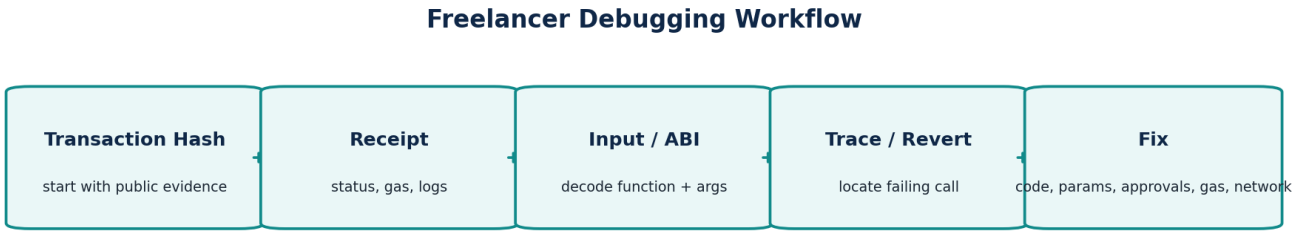
Important distinction

Transaction object tells you what was submitted. Receipt tells you what happened after inclusion/execution. Debugging usually needs both.

25. Logs vs return values

A transaction caller does not receive a Solidity return value later from the mined transaction in the same way an off-chain eth_call returns data immediately. For state-changing transactions, applications typically inspect the receipt, logs, subsequent state reads, and traces when necessary.

55-60 min - Practical Transaction Debugging Lab



Lab A - Successful contract call

10. Open a successful Ethereum contract-interaction transaction in an explorer.
11. Record: hash, block, from, to, nonce, value, gas limit, gas used and fee.
12. Identify the decoded function name if available.
13. Inspect token transfers and event logs.
14. Answer: what state-changing action did the transaction attempt, and what public evidence suggests success?

Lab B - Failed transaction

15. Open a failed transaction.
16. Confirm it was actually included in a block and has a receipt.
17. Inspect input/calldata and decoded function.
18. Look for a displayed revert reason or error data.
19. If trace tools are available, identify the call frame that reverted.
20. Write one hypothesis and one piece of evidence supporting it.

Rule for professionals

Evidence first, hypothesis second. A transaction hash is usually more useful than a screenshot saying “Transaction failed.”

Transaction Analysis Worksheet

Field	Your observation	Why it matters
Network		Chain mismatch is common
Hash		Primary reference
Status		Success/failure/inclusion
Block		Execution context
From		Who authorized
To / Contract		Target
Nonce		Sequence / replacement clues
Value		Native ETH attached
Function		Intent
Input / calldata		Actual encoded request
Gas limit		Execution cap
Gas used		Actual work
Fee		Cost
Token transfers		Asset movement
Logs/events		Application evidence
Revert reason / trace		Failure location

One-sentence conclusion:

_____

What additional evidence would you request from the client?

_____

Developer View - JSON-RPC Reads

Explorers are convenient, but applications normally retrieve Ethereum data through JSON-RPC. You do not need to memorize every method today; understand the pattern: request method + parameters -> node response.

```
POST /
{
  "jsonrpc": "2.0",
  "method": "eth_getTransactionByHash",
  "params": ["0xTRANSACTION_HASH"],
  "id": 1
}
```

Useful methods to recognize

Method	Purpose
eth_chainId	Identify current chain
eth_getBalance	Read ETH balance
eth_getTransactionCount	Read account nonce / transaction count for a block tag
eth_getTransactionByHash	Fetch submitted transaction fields
eth_getTransactionReceipt	Fetch execution result + logs
eth_call	Simulate/read a call without publishing a state-changing transaction
eth_estimateGas	Estimate gas for a proposed call
eth_getLogs	Query event logs

Freelancer payoff

When a frontend says “MetaMask is broken,” test the same read through RPC. Separating wallet, frontend, RPC and contract layers makes troubleshooting much faster.

Upwork / LinkedIn Client Scenarios

Scenario 1 - “Transaction is pending forever”

- Check whether it is actually pending or already replaced/dropped.
- Check sender nonce sequence for earlier pending transactions.
- Compare transaction fee settings with current network conditions.
- Check whether a newer transaction with the same nonce replaced it.
- Do not tell the client to expose their private key; public tx/account data is enough for initial diagnosis.

Scenario 2 - “Contract function reverted”

- Confirm correct chain and contract address.
- Decode function + parameters from calldata.
- Read prerequisites: balances, allowances, roles, deadlines, pause state, contract configuration.
- Inspect revert data and call trace.
- Reproduce on a fork/test environment before proposing a fix.

Scenario 3 - “Token transfer says success, but balance is wrong”

- Verify the token contract address, not just the token symbol.
- Inspect Transfer event logs and decimals.
- Check recipient address exactly.
- Check whether the UI is on the correct chain and reading the correct token contract.
- Distinguish transaction success from application display/cache/indexing problems.

Proposal language

A strong freelancer says: “Send the network, transaction hash, contract address, expected result and relevant frontend/RPC error. I can trace the call and identify whether the issue is input, contract logic, allowance, gas, network or UI.”

Common Beginner Mistakes

Mistake	Correct mental model
“Solidity runs directly on Ethereum.”	Solidity is compiled; the EVM executes bytecode.
“ABI is inside the contract and executes functions.”	ABI is an interface schema used by tools to encode/decode data.
“Gas limit is the fee.”	Gas limit is an execution cap; fee depends on gas used and per-gas price.
“A failed transaction changed nothing and cost nothing.”	Reverted state changes roll back, but gas can still be consumed.
“Events are storage variables.”	Events create receipt logs; they are not persistent contract storage.
“Input data is readable text.”	It is encoded bytes; ABI-aware tools decode it.
“A contract can send itself a normal wallet transaction whenever it wants.”	Contract execution occurs when triggered by a transaction/call path.
“Same token symbol means same token.”	Contract address + chain identify the token.

Checkpoint

If students can explain a failed transaction using: sender -> nonce -> target -> calldata -> EVM path -> storage/calls -> revert/receipt, Day 3 is working.

Day 3 Quiz & Interview Practice

10 quick questions

1. What is the difference between an EOA and a contract account?
2. What is the purpose of an account nonce?
3. What is calldata?
4. What does the EVM execute: Solidity source code or bytecode?
5. Which data area persists across transactions: memory or storage?
6. What is the difference between gas limit and gas used?
7. What does an ABI describe?
8. How is a typical Solidity function selector derived?
9. Where are Solidity event logs recorded?
10. What is the difference between a transaction object and a transaction receipt?

Answer key

1. EOA is traditionally private-key controlled and can initiate signed transactions; a contract account executes bytecode and has persistent storage.
2. It sequences transactions from the account and prevents replay at the same sequence position.
3. Read-only external call input bytes, commonly containing selector + ABI-encoded arguments.
4. EVM bytecode.
5. Storage.
6. Gas limit is a maximum allowance; gas used is actual consumed execution work.
7. The contract interface schema for encoding calls and decoding returns/logs/errors.
8. First four bytes of Keccak-256 of the canonical function signature.
9. In transaction receipt logs.
10. Transaction = submitted intent/fields; receipt = result after inclusion/execution.

Day 3 Homework - 45 to 60 Minutes

Task 1 - Transaction analysis

Choose three public Ethereum transactions: one native ETH transfer, one ERC-20 transfer, and one contract call. Complete the worksheet for each. Do not use a transaction containing your personal sensitive information.

Task 2 - Decode intent

- For the contract call, write the function name and arguments shown by the explorer.
- Copy the raw input data and identify the first 4-byte selector.
- Explain in one paragraph how the ABI allowed the explorer to display human-readable parameters.

Task 3 - EVM data locations

Create your own table with four rows: stack, memory, storage, calldata. For each, write: persistence, mutability, typical use and one Solidity example.

Task 4 - Freelancer response

Write a 5-sentence response to this client: “Our staking transaction fails in MetaMask. Can you fix it?” Your response must request public evidence and describe the layers you will check without requesting private keys.

Task 5 - Prepare for Day 4

- Install/verify Git, Node.js and VS Code.
- Create a GitHub account/repository if needed.
- Be ready to use Remix plus one local Solidity toolchain.
- Understand the terms RPC endpoint, testnet, compiler and package manager.

Pass standard

Before Day 4, you should be able to look at an explorer transaction and explain: who authorized it, which network it targeted, what contract/function it called, what gas was consumed, whether it succeeded, and where to look next if it failed.

Day 3 Printable Cheat Sheet

```
ETHEREUM MENTAL MODEL

EOA signs transaction
      |
      v
Transaction: chainId | nonce | to | value | data | gas | fee | signature
      |
      v
EVM executes contract bytecode
      |
      +--> Stack     : temporary 256-bit operand workspace
      +--> Memory    : temporary mutable bytes
      +--> Storage   : persistent contract state
      +--> Calldata  : read-only external input
      |
      v
Receipt: status | gasUsed | logs | block context
      |
      v
New world state (if successful)
```

Term	One-line meaning
EOA	Private-key-authorized account that can initiate signed transactions
Contract account	Address with executable EVM bytecode and persistent storage
Nonce	Transaction sequence position for sender
Calldata	Read-only bytes supplied to an external call
Bytecode	Instructions executed by the EVM
Opcode	One EVM instruction
ABI	Schema for encoding/decoding contract interaction
Selector	First 4 bytes identifying a standard ABI function call
Gas	Meter for EVM execution work
Storage	Persistent contract state
Memory	Temporary execution workspace
Event/log	Receipt record for application indexing/observation
Receipt	Execution result for an included transaction
Revert	Execution failure that rolls back state changes in the reverted path

Day 4 preview: Professional Blockchain Development Environment - Remix, VS Code, Git/GitHub, Node.js, Foundry/Hardhat, RPC endpoints and testnet deployment workflow.

Technical References & Further Reading

Use primary documentation when details matter. The following references informed the lecture and are suitable for deeper study:

Ethereum Developers - Transactions
https://ethereum.org/developers/docs/transactions/

Ethereum Developers - Ethereum Virtual Machine (EVM)
https://ethereum.org/developers/docs/evm/

Ethereum Developers - Accounts
https://ethereum.org/developers/docs/accounts/

Ethereum Developers - JSON-RPC API
https://ethereum.org/developers/docs/apis/json-rpc/

Solidity Documentation - Contract ABI Specification
https://docs.soliditylang.org/en/latest/abi-spec.html

Solidity Documentation - Types / Data Locations
https://docs.soliditylang.org/en/latest/types.html

Solidity Documentation - Introduction to Smart Contracts
https://docs.soliditylang.org/en/latest/introduction-to-smart-contracts.html

Solidity Documentation - Structure of a Contract / Events
https://docs.soliditylang.org/en/latest/structure-of-a-contract.html

Scope note

This Day 3 lecture deliberately simplifies several protocol internals. Later lessons will add Solidity implementation details, testing, storage layout, security, RPC integration, DeFi execution and transaction tracing.